



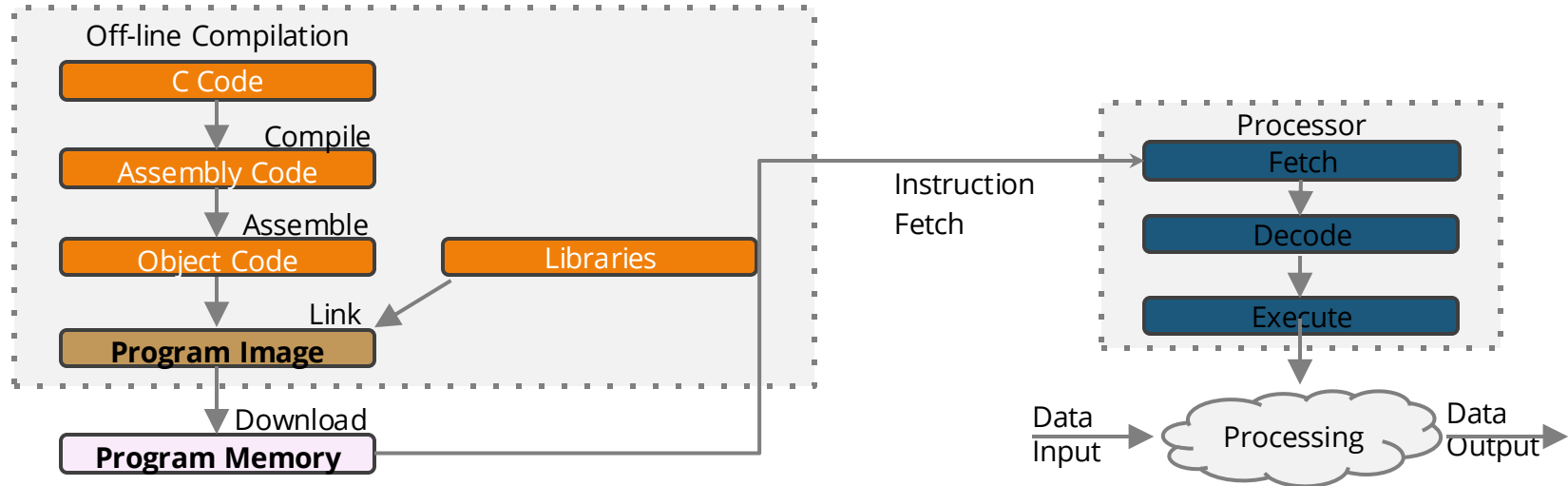
Embedded Software for the Internet Of Things

Embedded Program Development

Assoc. Prof. Kasim Sinan Yildirim
University of Trento, Italy

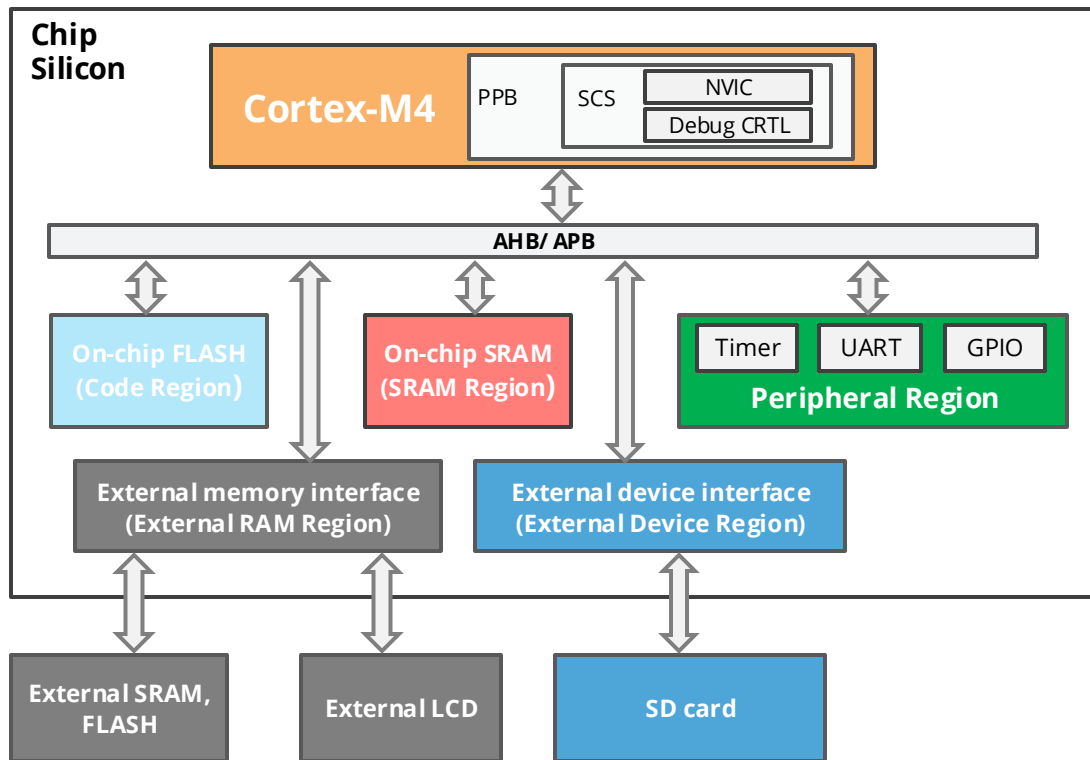
Typical Program-generation Flow

- Compile -> Assemble -> Link -> Download
 - The generated **executable file** (or **program image**) is stored in the program memory (normally an **on-chip flash memory**), to be fetched by the processor



Typical program-generation flow

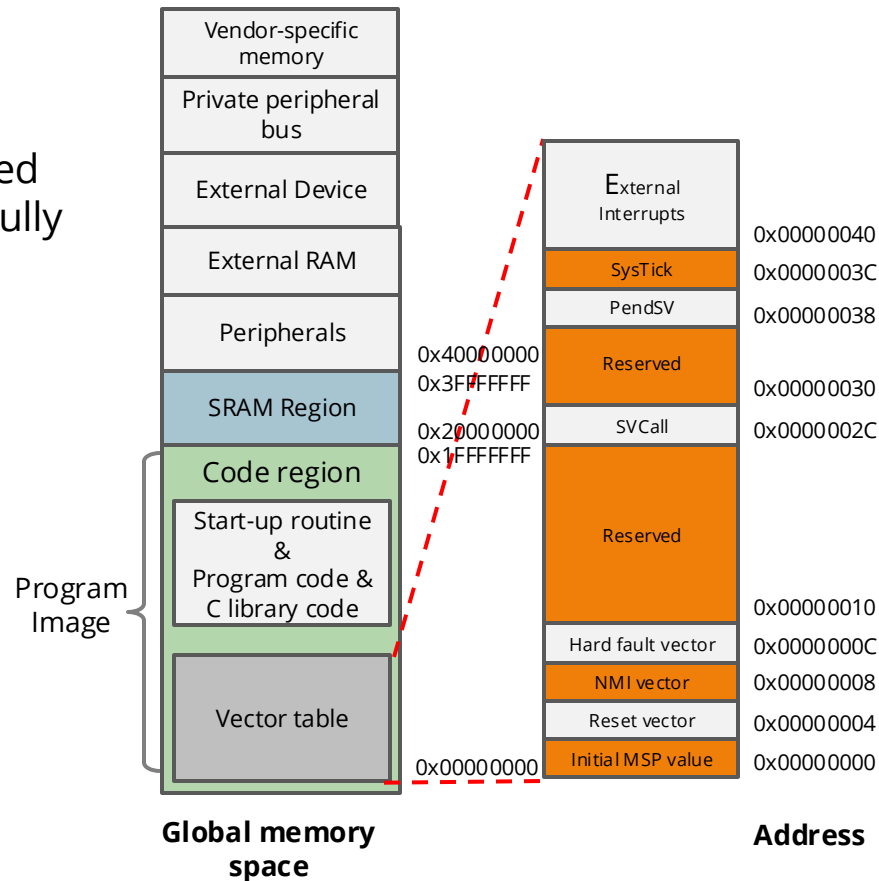
Cortex-M4 memory map example



Vendor specific Memory	0xFFFFFFFF 0xE0100000
Private Peripheral Bus (PPB)	0xE00FFFFFFF 0xE0000000 0xDFFFFFFF
External device	0xA0000000 0x9FFFFFFF
External RAM	0x60000000 0x5FFFFFFF
Peripherals	0x40000000 0x3FFFFFFF
SRAM	0x20000000 0x1FFFFFFF
Code	0x00000000

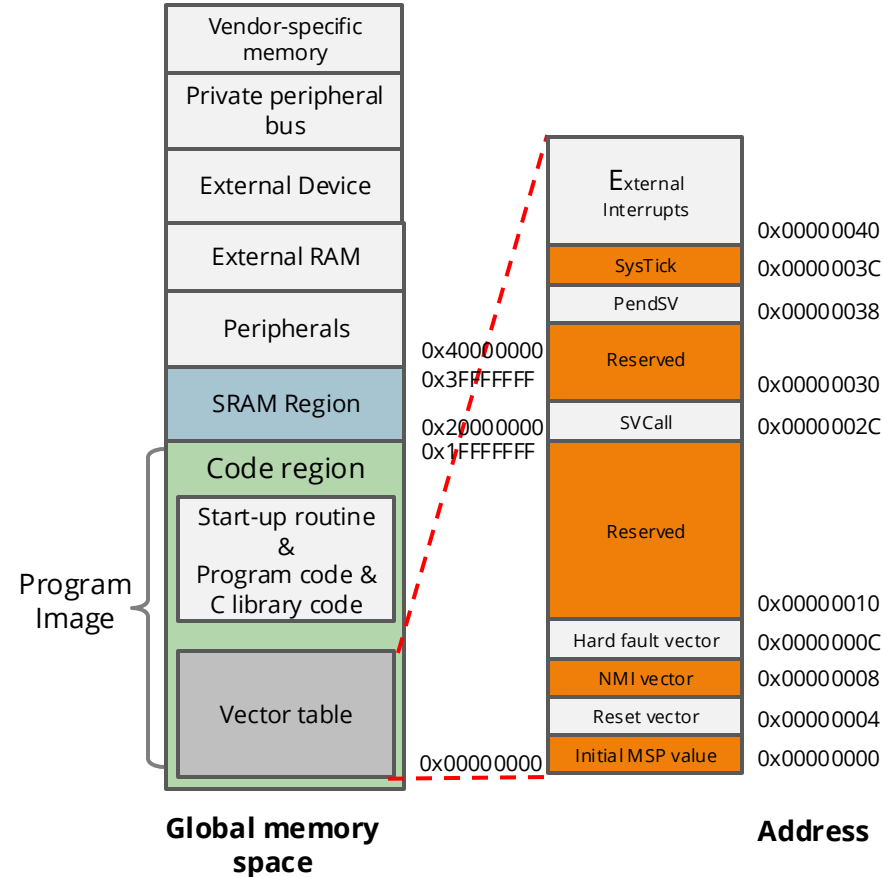
Cortex-M4 Program Image

- What is a **program image**?
 - The program image (sometimes also called the **executable file**) refers to a piece of fully integrated code that is ready to execute.



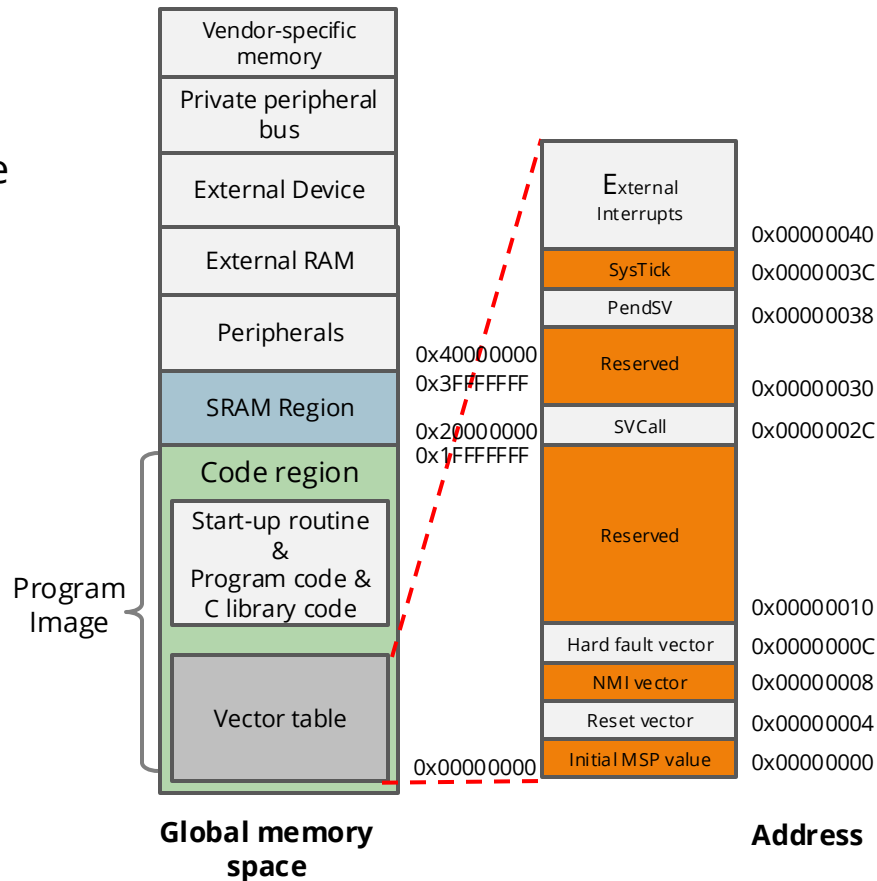
Cortex-M4 Program Image

- In the Cortex-M4, the program image includes:
 - Vector table
 - C start-up routine
 - Program code
 - C library code



Cortex-M4 Program Image

- **Program code**
 - Instructions and data generated from the application program.
- **C library code**
 - Object codes inserted into the program image by linkers



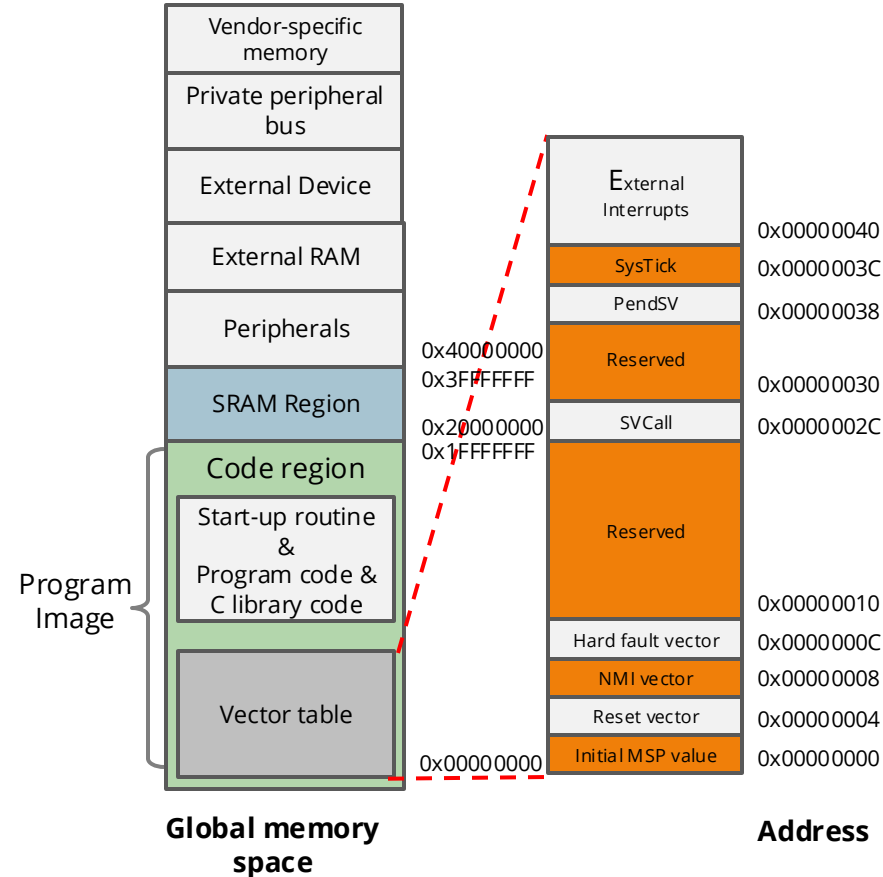
Cortex-M4 Program Image

- **Vector table**

- Contains the starting addresses of interrupt vectors and the value of the **main stack point (MSP)**

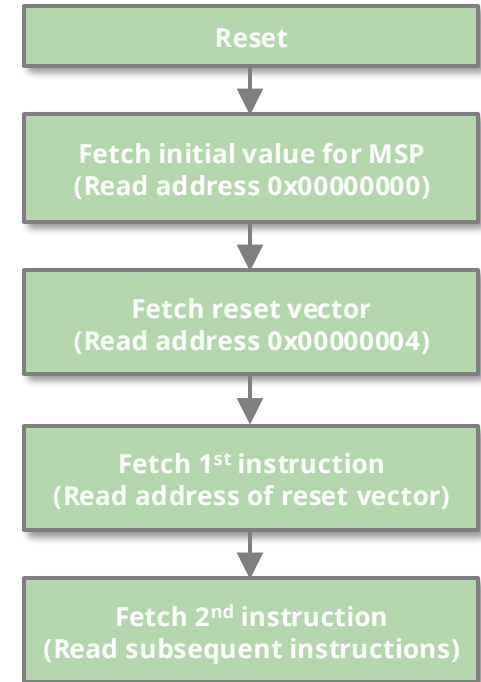
- **C Start-up code**

- Used to set up data memory and the initialization of values for global data variables
- Inserted by the **compiler/linker** automatically, e.g., labeled as **'__start'** by the GNU C compiler



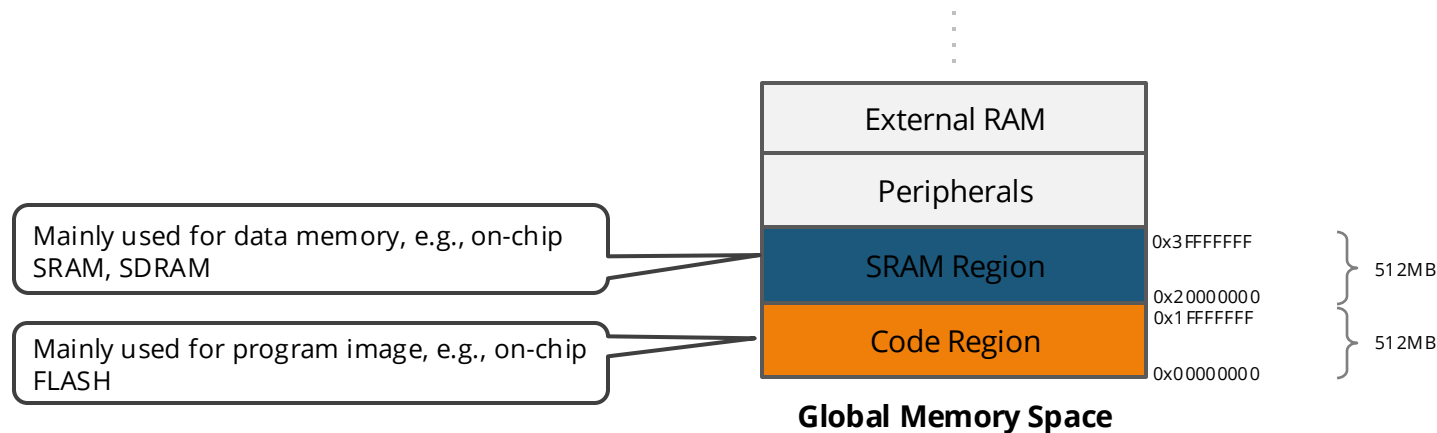
Systems Initialization (in ARM Cortex Processors)

- After reset, the processor:
 - First reads the initial MSP value
 - Then reads the **reset vector**
 - Then branches to the start of the program execution address (**reset handler**)
 - Subsequently executes program instructions



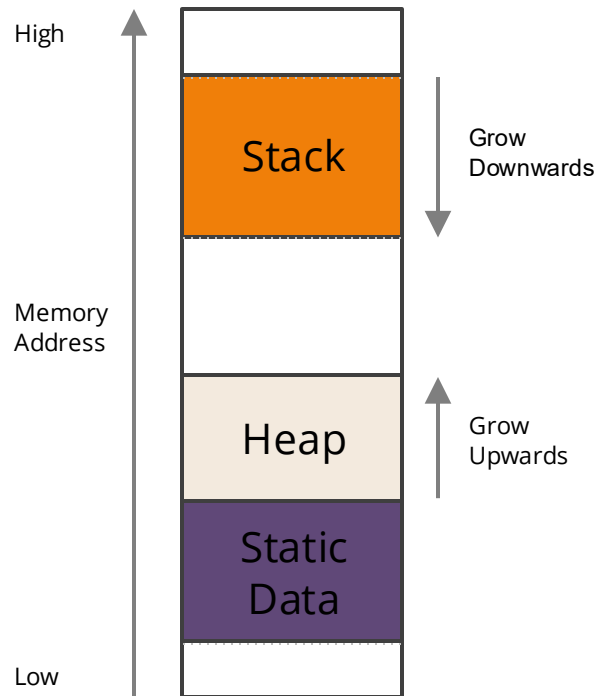
Program Image in Global Memory

- The **program image** is stored in the code region in global memory
 - Up to 512 MB memory space range from 0x00000000 to 0x1FFFFFFF
 - Usually implemented on **non-volatile memory**, such as **on-chip FLASH memory**
- Normally **separated** from program data, which is allocated in the **SRAM** region (or data region)



How is Data Stored in SRAM?

- Typically, the data can be divided into three sections: **static data**, **stack**, and **heap**
 - **Static data:** global variables and static variables
 - **Stack:** local variables, parameter passing in function calls, registers saving during exceptions, etc.
 - **Heap:** pieces of memory spaces that are dynamically reserved by function calls, such as 'alloc()' and 'malloc ()'
 - *Generally not preferred in embedded systems*



What Memory Does a Program Need?

- Can the information change?
 - No? Put it in **read-only nonvolatile memory**
 - *Instructions, Constant strings, Constant operands, Initialization values...*
 - Yes? Put it in **volatile (read/write memory)**
 - *Variables, Intermediate computations, Return address...*

```
int a, b;  
const char c=123;  
int d=31;  
void main(void) {  
    int e;  
    char f[32];  
    e = d + 7;  
    a = e + 29999;  
    strcpy(f,"Hello!");  
}
```

What Memory Does a Program Need?

- How long does the data need to exist? **Reuse** memory if possible.
 - Statically allocated
 - *Exists from program start to end*
 - *Each variable has its own **fixed location***
 - *Space is not reused*
 - Automatically allocated
 - *Exists from function start to end*
 - *Space can be **reused***
 - Dynamically allocated
 - *Explicit allocation to explicit **de-allocation***
 - *Space can be reused*

```
int a, b;  
const char c=123;  
int d=31;  
void main(void) {  
    int e;  
    char f[32];  
    e = d + 7;  
    a = e + 29999;  
    strcpy(f,"Hello!");  
}
```

Executable Image Sections

- Executable image contains **initialized data** and **uninitialized data** sections:
 - **.data** and **.sdata**
 - *contain the initial values for the global and static variables*
 - **.bss** and **.sbss**
 - *uninitialized (or zero initialized) data sections (their content is empty)*
 - **.const**
 - *constant data is part of this section which is read-only.*

Program Memory Use

RAM

Zero-Initialized Data .bss and .sbss
Initialized Data
Stack
Heap Data

```
int a, b;  
const char c=123;  
int d=31;  
void main(void) {  
    int e;  
    char f[32];  
    e = d + 7;  
    a = e + 29999;  
    strcpy(f,"Hello!");  
}
```

Flash ROM

Constant Data .const
Initialization Data .data and .sdata
Startup and Runtime Library Code .text
Program .text

Program Memory Use

RAM

Zero-Initialized Data .bss and .sbss
Initialized Data
Stack
Heap Data

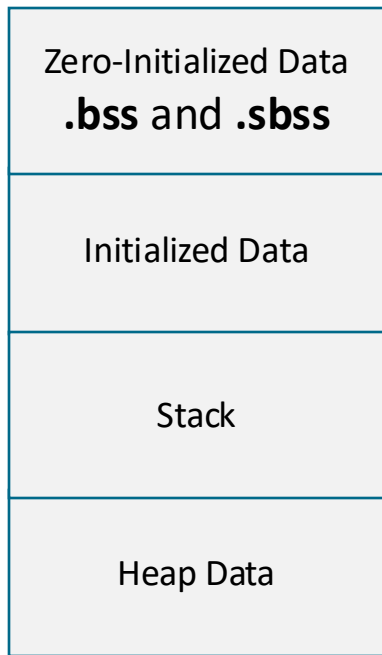
```
int a, b;  
const char c=123;  
int d=31;  
void main(void) {  
    int e;  
    char f[32];  
    e = d + 7;  
    a = e + 29999;  
    strcpy(f, "Hello!");  
}
```

Flash ROM

Constant Data .const
Initialization Data .data and .sdata
Startup and Runtime Library Code .text
Program .text

Program Memory Use

RAM



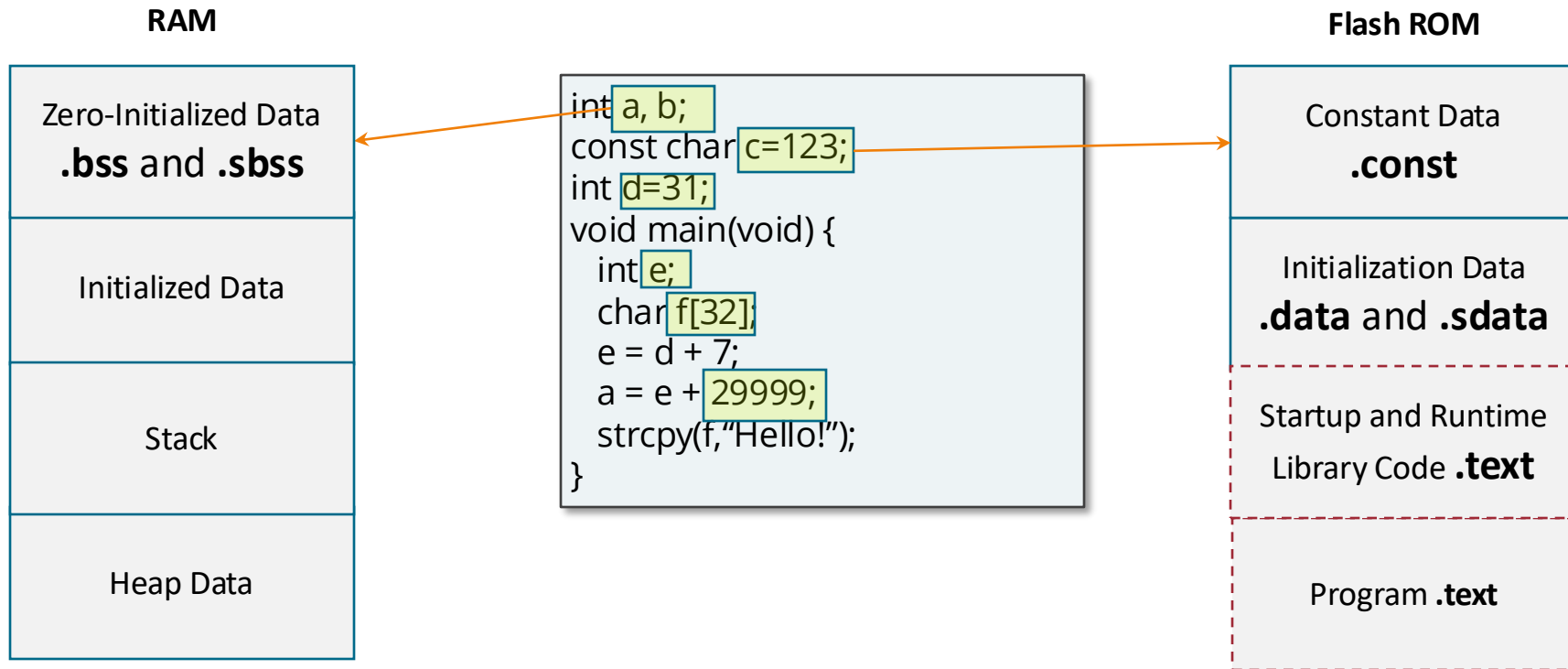
```
int a, b;  
const char c=123;  
int d=31;  
void main(void) {  
    int e;  
    char f[32];  
    e = d + 7;  
    a = e + 29999;  
    strcpy(f, "Hello!");  
}
```



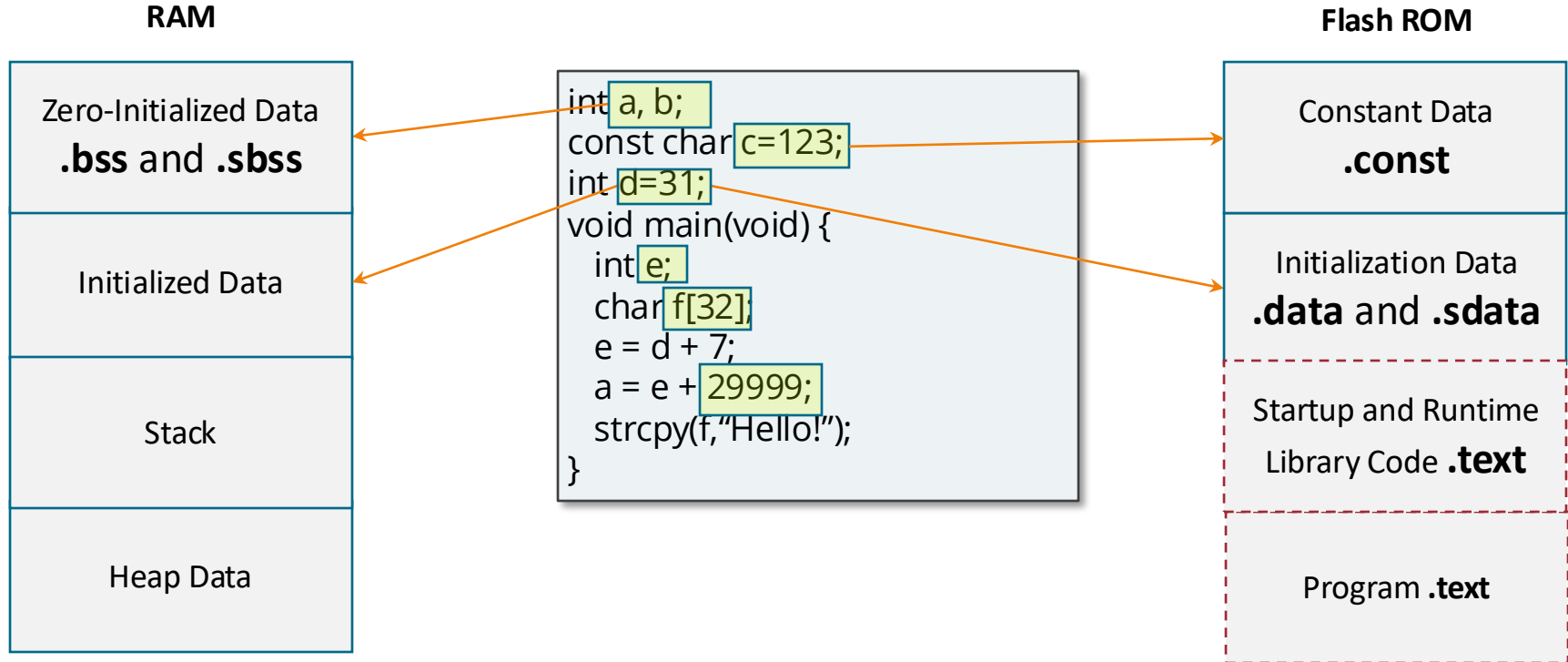
Flash ROM



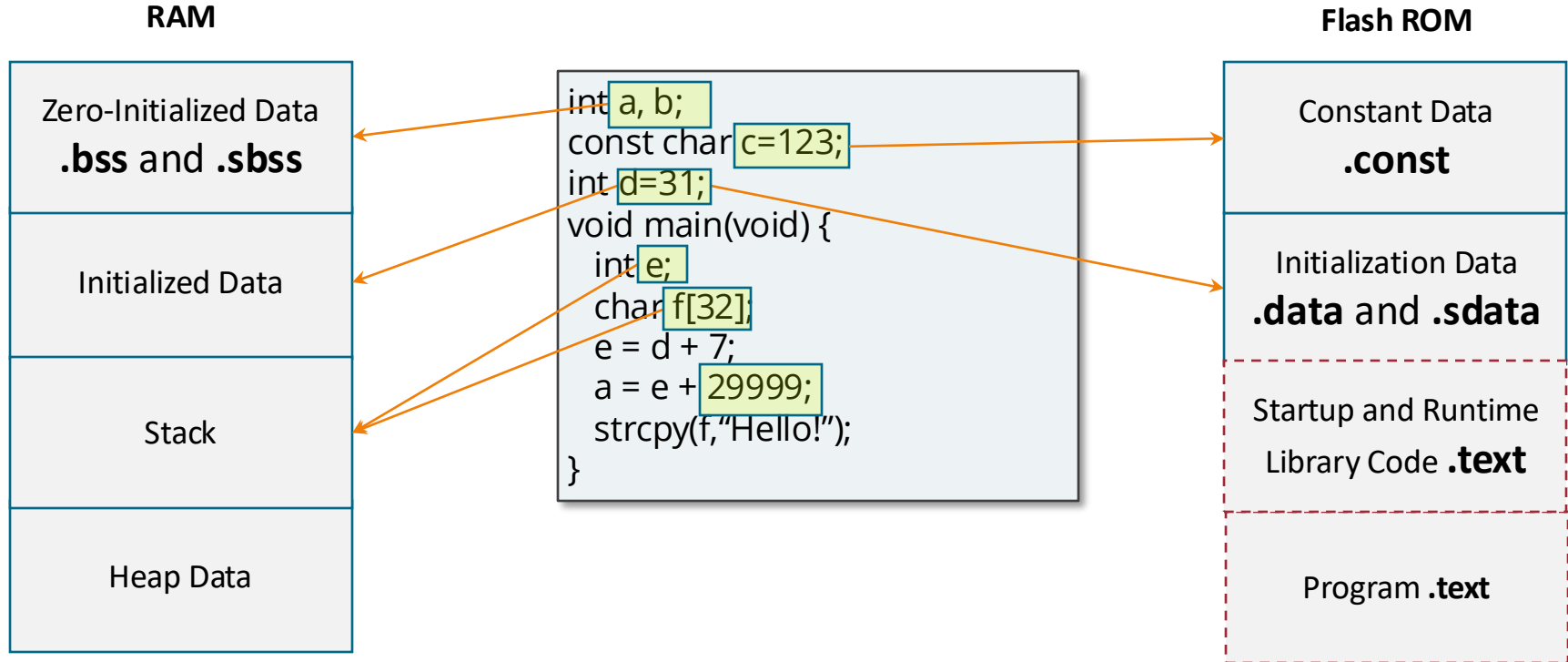
Program Memory Use



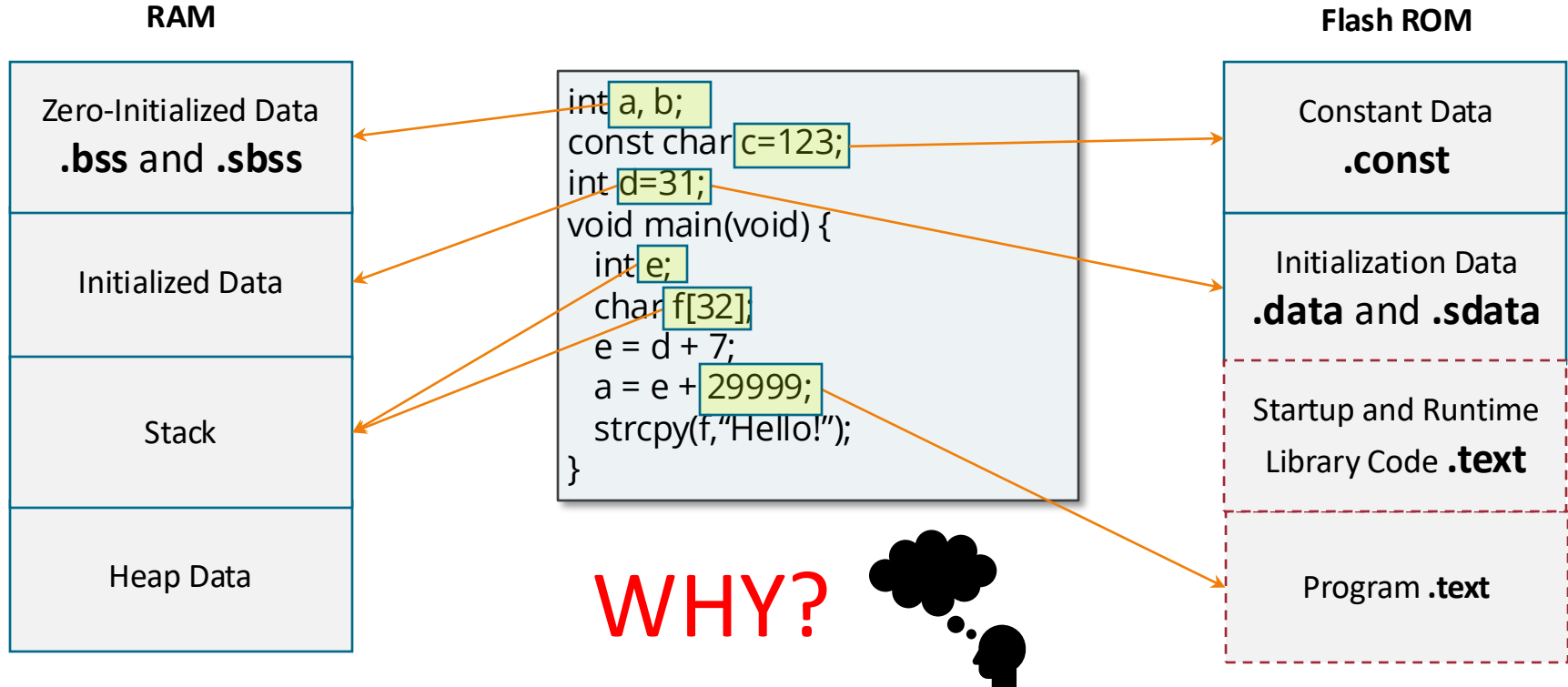
Program Memory Use



Program Memory Use



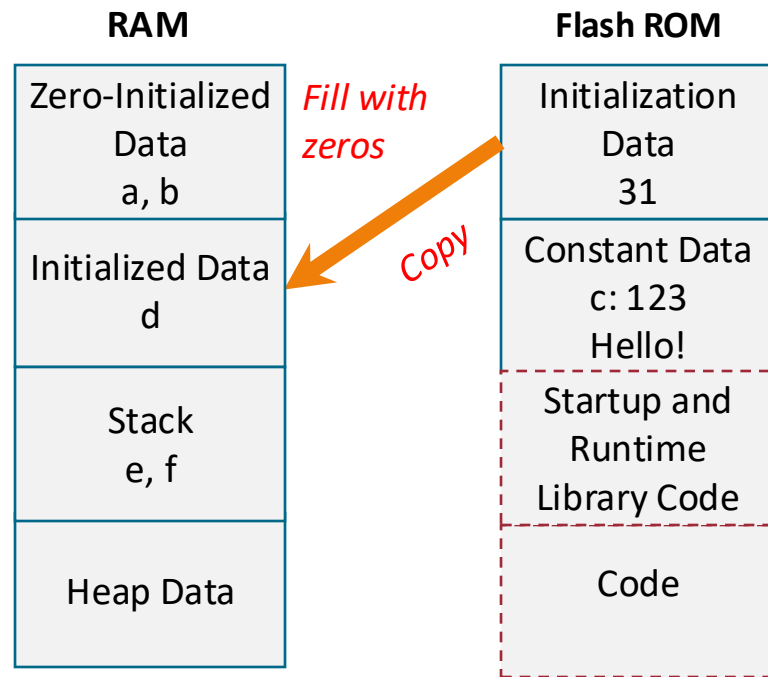
Program Memory Use



C Run-Time Start-Up Module

- After reset, MCU must:
 - Initialize hardware ,...
- Initialize C or C++ runtime environment
 - Set up heap memory, Initialize variables

```
int a, b;  
const char c=123;  
int d=31;  
void main(void) {  
    int e;  
    char f[32];  
    e = d + 7;  
    a = e + 29999;  
    strcpy(f,"Hello!");  
}
```



Loading Program Image

- Transferring an executable image built for a target embedded system from the **host** onto the **target**
 - The entire executable image is burned into the ROM or flash memory using **special equipment**.
- Each time the code changes due to bugs or code addition:
 - Reprogram the ROM or the flash memory using special equipment.
 - e.g., *JTAG interface* (JTAG - Joint Test Action Group)

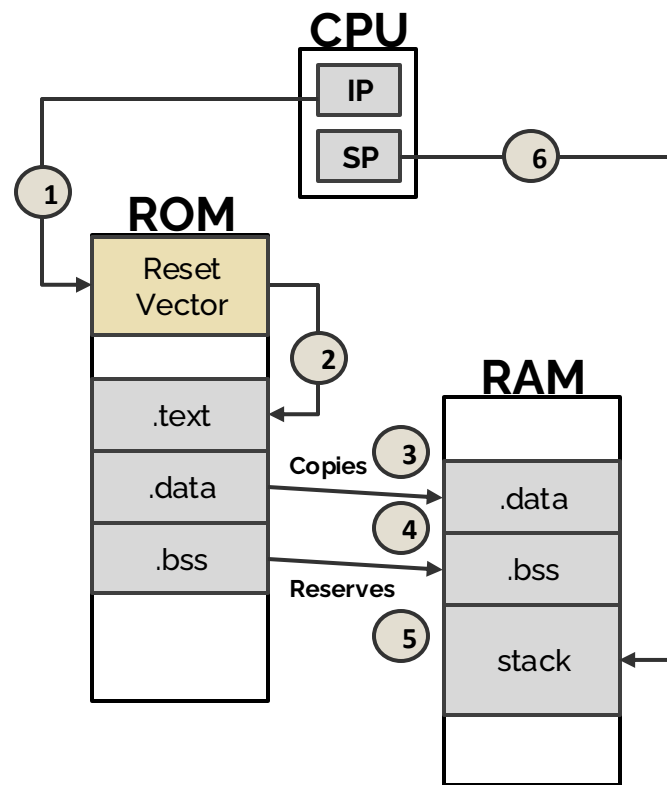


An Example Booting Procedure

- After powered on:
 - Fetch and execute code from a predefined and hard-wired address offset.
 - The code contained at this memory location is called the **reset vector**.
- The reset vector is usually a jump instruction into the real initialization code, called the **bootstrap code**
 - The reason for jumping to another part of memory is to keep the reset vector small.

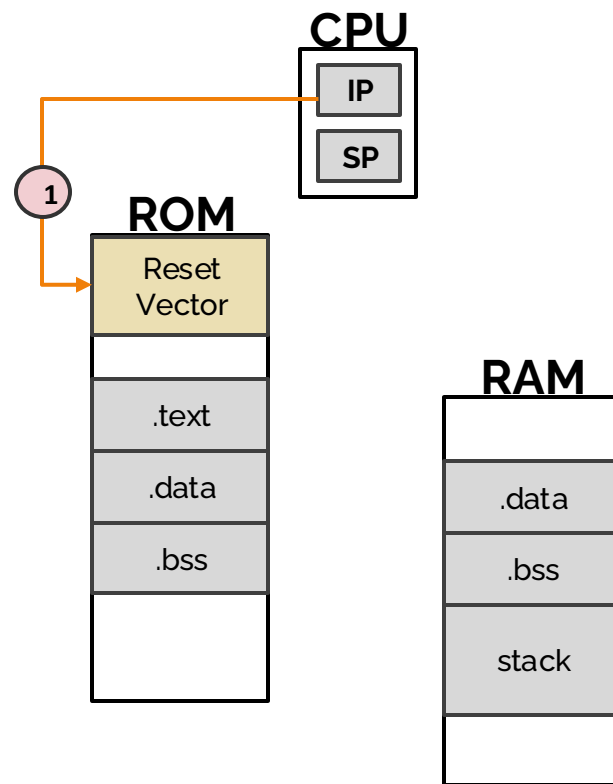
An Example Booting Procedure

- Two CPU registers are of concern:
 - **Instruction Pointer (IP) register**
 - *points to the next instruction (code in the .text section)*
 - **Stack Pointer (SP) register**
 - *points to the next free address in the stack.*
- The SP must be set appropriately at start-up.
 - The stack is created from a space in RAM



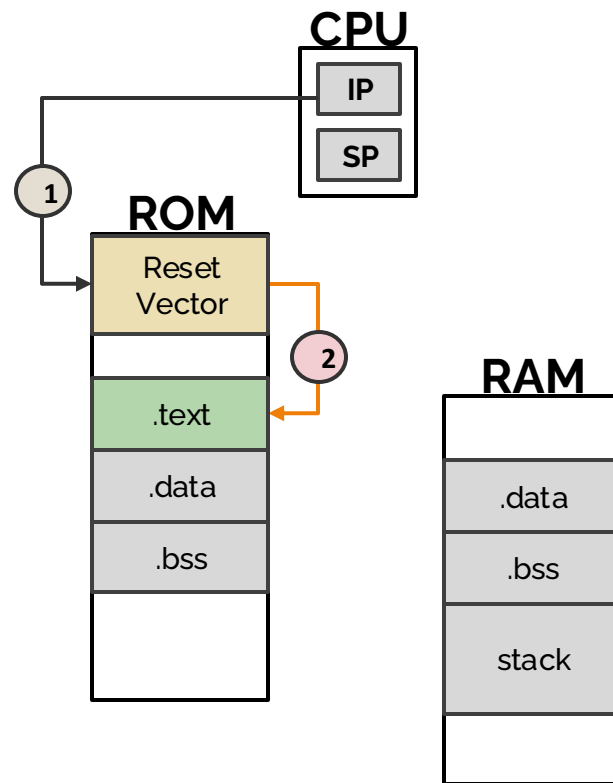
An Example Booting Procedure

- (1) IP is **hardwired** to execute the first instruction in memory
 - the **reset vector**



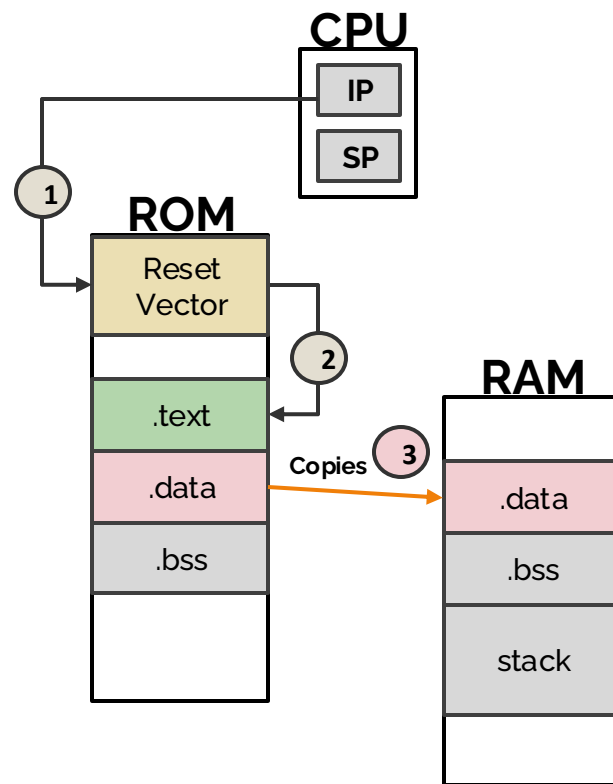
An Example Booting Procedure

- (1) IP is **hardwired** to execute the first instruction in memory
 - the **reset vector**
- (2) The reset vector jumps to the first instruction of the `.text` section in ROM



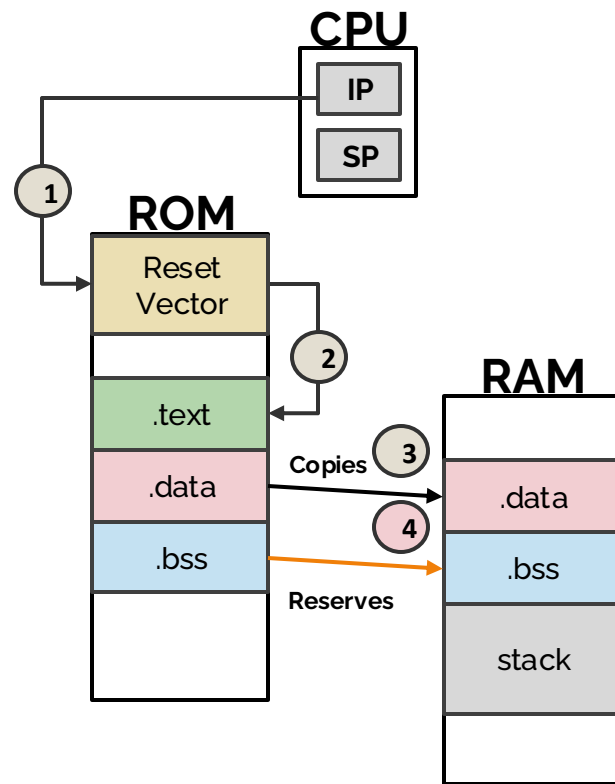
An Example Booting Procedure

- (1) IP is **hardwired** to execute the first instruction in memory
 - the **reset vector**
- (2) The reset vector jumps to the first instruction of the **.text** section in ROM
- (3) The **.data** section of the boot image is copied **into RAM**
 - it is both readable and writable.



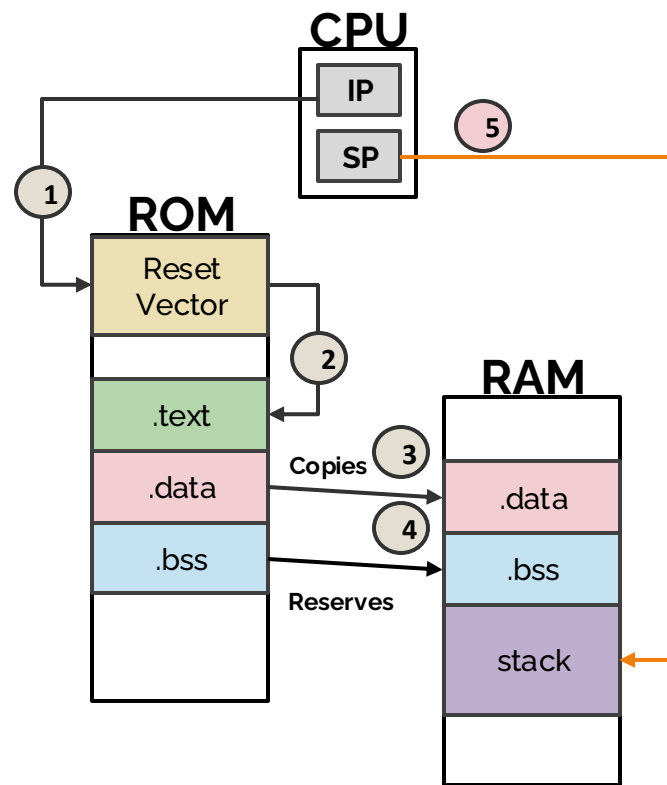
An Example Booting Procedure

- (4) Space is reserved in RAM for the .bss section
 - Nothing to transfer because the content for the .bss section is empty.



An Example Booting Procedure

- (4) Space is reserved in RAM for the .bss section
 - Nothing to transfer because the content for the .bss section is empty.
- (5) Stack space is reserved in RAM. The SP register is set to point to the beginning of the newly created stack.

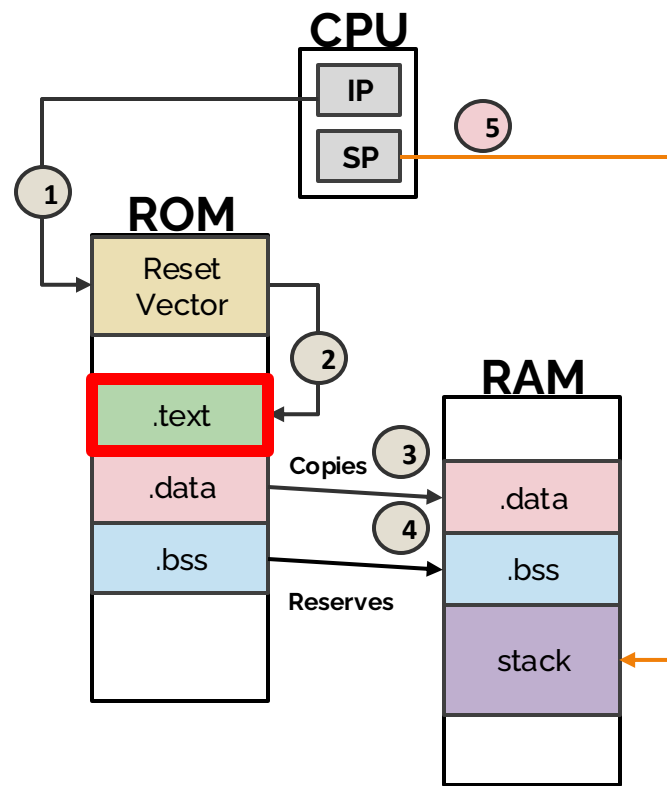


An Example Booting Procedure

- (4) Space is reserved in RAM for the .bss section
 - Nothing to transfer because the content for the .bss section is empty.
- (5) Stack space is reserved in RAM. The SP register is set to point to the beginning of the newly created stack.

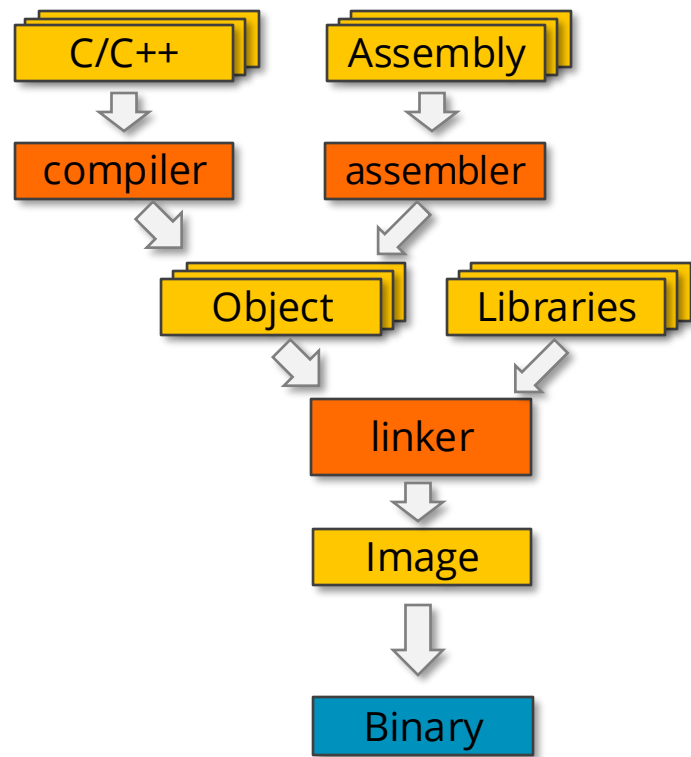
At this point, the boot completes:

The CPU continues to execute the code in the .text section



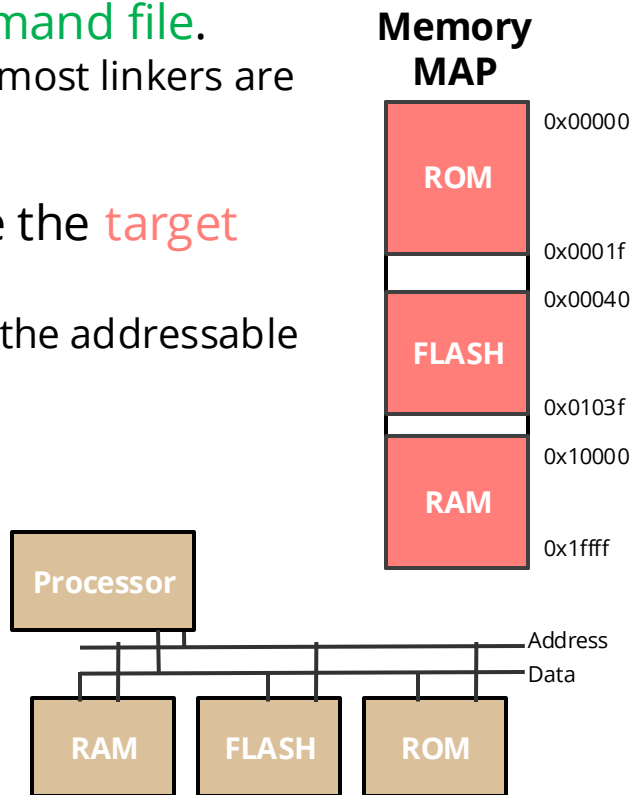
Linker Map File

- The linker creates a **single executable image** for the target embedded system.
 - Merge sections from the different object files into program segments
- **Linker commands** (called linker directives) control how the linker combines the sections and allocates the segments into the target system.



Linker Command File

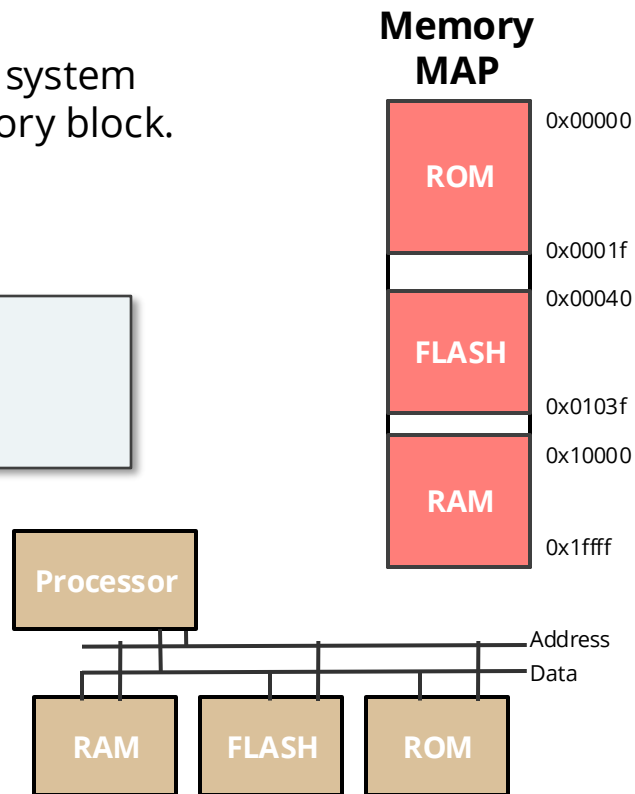
- The linker directives are kept in **the linker command file**.
 - Two of the more common directives supported by most linkers are **MEMORY** and **SECTION**.
- The **MEMORY** directive can be used to describe the **target system's memory map**.
 - An embedded developer needs to be familiar with the addressable physical memory on a target system.



Linker Command File

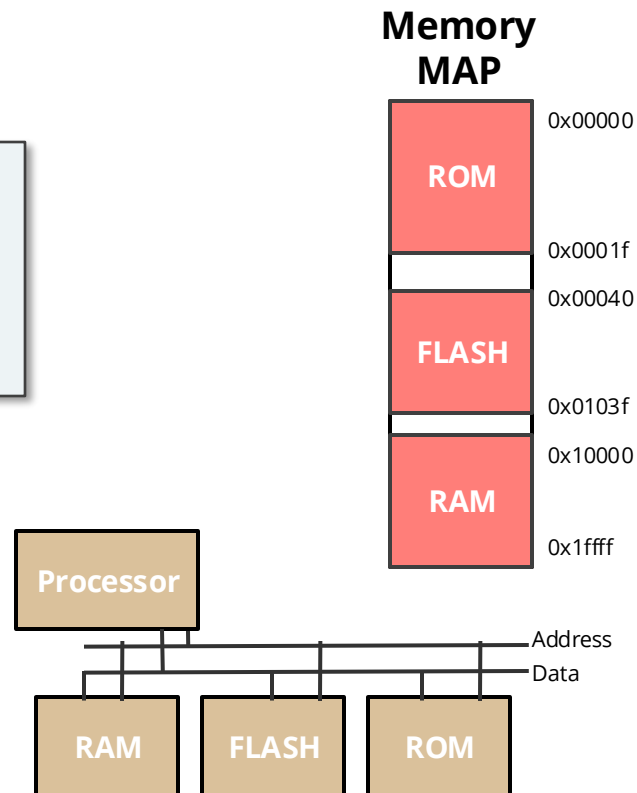
- The **MEMORY** directive defines:
 - the types of physical memory present in the target system
 - the address range occupied by each physical memory block.
- Format:

```
MEMORY {  
    area-name : org = start-address, len = number-of-bytes  
    ...  
}
```



Linker Command File

```
MEMORY {  
    ROM: origin = 0x0000h, length = 0x0020h  
    FLASH: origin = 0x0040h, length = 0x1000h  
    RAM: origin = 0x1000h, length = 0x10000h  
}
```



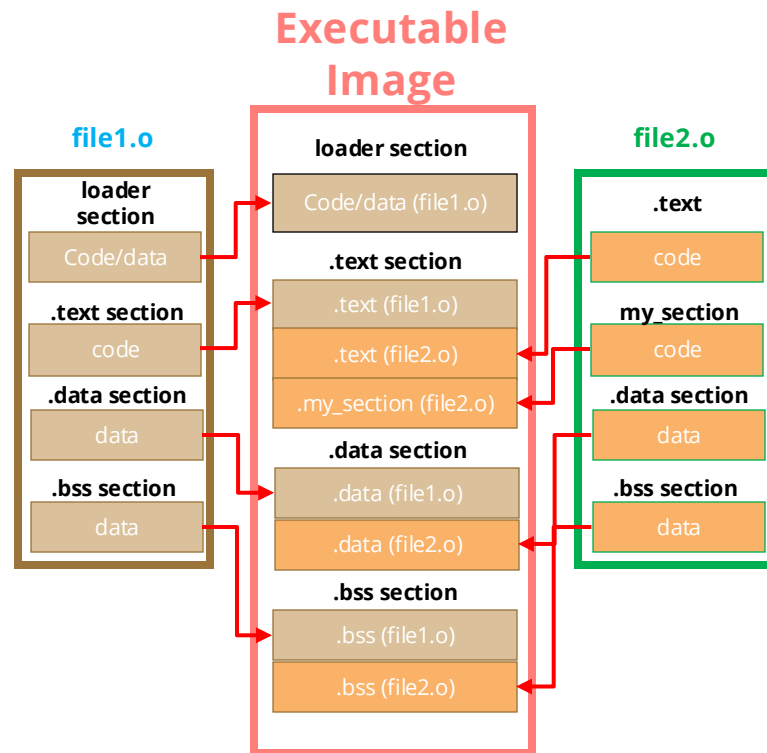
Linker Command File

- The **SECTION** directive tells the linker:
 - which input sections are **to be combined** into which output section
 - which output sections are to be **grouped together** and allocated in contiguous memory
 - where to place each section

```
SECTION {  
    output-section-name : { contents } > area-name  
    ...  
    GROUP {  
        [ALIGN(expression)]  
        section-definition  
        ...  
    } > area-name  
}
```

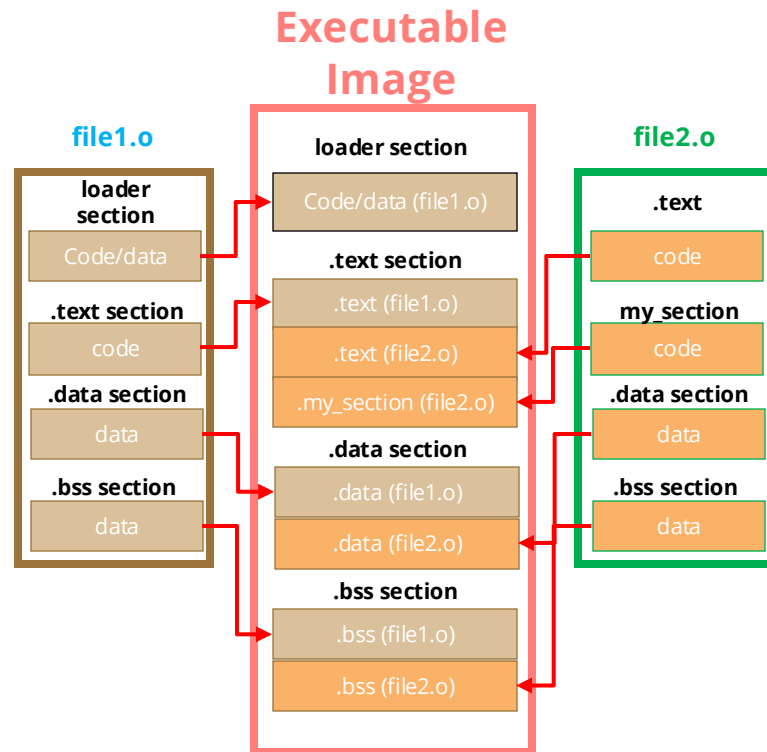
Linker Command File

- Example:
 - two object files generated by a compiler or assembler (file1.o and file2.o)
 - three default sections (.text, .data, and .bss)
 - two developer-specified sections (loader and my_section)



Linker Command File

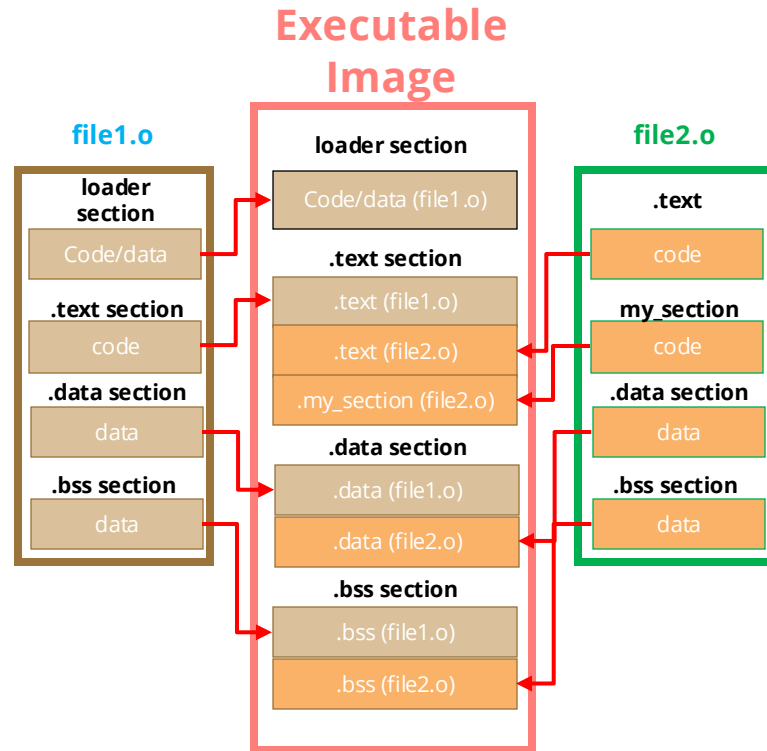
```
SECTION {  
    .text :  
    {  
        my_section  
        *(.text)  
    }  
    loader : > FLASH  
    GROUP ALIGN (4) :  
    {  
        .data : {}  
        .bss : {}  
    } > RAM  
}
```



Linker Command File

```
SECTION {  
    .text :  
    {  
        my_section  
        *(.text)  
    }  
    loader : > FLASH  
    GROUP ALIGN (4) :  
    {  
        .data : {}  
        .bss : {}  
    } > RAM  
}
```

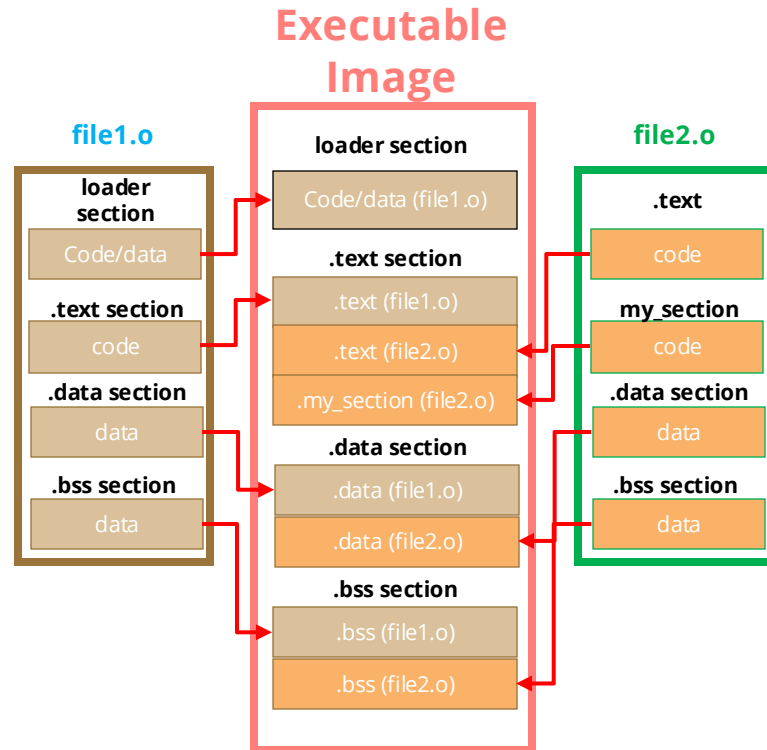
combine **my_section** and the **.text** sections from all object files into the final output **.text** section



Linker Command File

```
SECTION {  
    .text :  
    {  
        my_section  
        *(.text)  
    }  
    loader : > FLASH  
    GROUP ALIGN (4) :  
    {  
        .data : {}  
        .bss : {}  
    } > RAM  
}
```

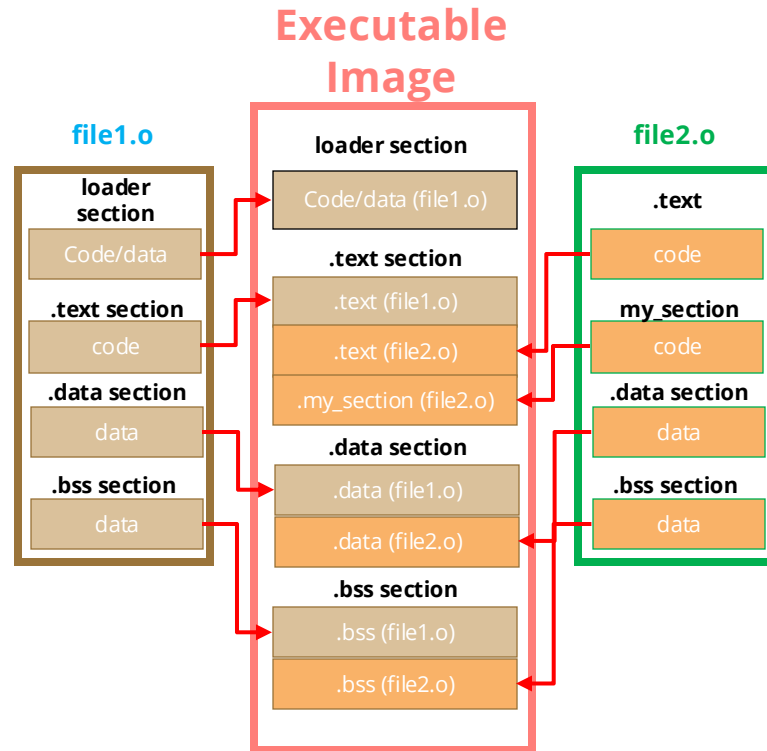
The loader section is placed into **flash memory**.



Linker Command File

```
SECTION {  
    .text :  
    {  
        my_section  
        *(.text)  
    }  
    loader : > FLASH  
    GROUP ALIGN (4) :  
    {  
        .data : {}  
        .bss : {}  
    } > RAM  
}
```

.data and **.bss** are grouped and allocated in **contiguous** physical RAM **aligned** on the 4-byte boundary



Linker Command File

```
MEMORY {  
    ROM: origin = 0x0000h, length = 0x0020h  
    FLASH: origin = 0x0040h, length = 0x1000h  
    RAM: origin = 0x1000h, length = 0x10000h  
}
```

